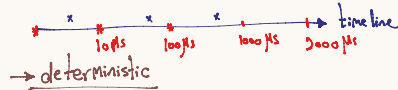


Interrupt

* some concept for SW Eng

1) Determinism

↳ in SW :: knowing what happening at every point in timeline



2) Responsiveness

↳ in SW → How fast you can respond to event.
↳ SW → High Responsiveness

* SW system type

1) Super loop → while(1)

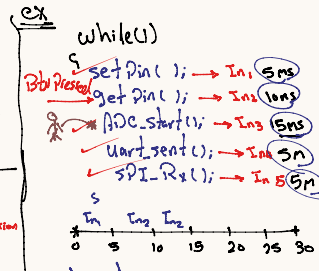
↳ the all project app running inside while(1)
↳ each instruction → order execute.
↳ this way → provide the high Determinism
↳ this system → low Responsiveness.

Advantage

- 1) High determinism
- 2) Simple SW
- 3) Minimal H.W

disadvantage

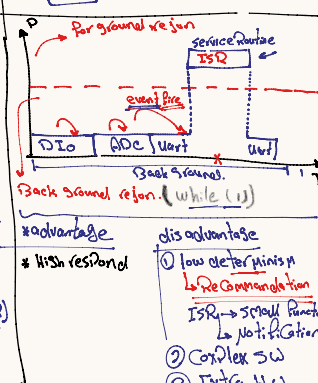
- 1) Low responsiveness
- 2) High Power Consumption



2) For Background system

↳ Divide Application to (Normal, High Priority)
↳ High Priority → Run in foreground
↳ Normal → Run in Background region

* Interrupt
1) when event fire we interrupt processor to execute the specific code.
2) Specific code called Interrupt service Routine (ISR)
3) we can't detect when the event fire



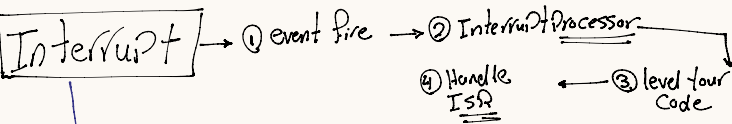
advantage

- 1) High respond

disadvantage

- 1) low determinism
↳ No Command
↳ ISR → small function
↳ No Notification
- 2) Complex SW
- 3) Extra H.W

3) Real time operating system (RTOS)



↳ 3 Condition to Interrupt happen.

- 1) Flag → event fire or not.
↳ Peripheral Interrupt Flag → Must be 1
- 2) Processor Enable for Interrupt concept
↳ Global Interrupt Enable → Must be 1
- 3) Peripheral → allow to Interrupt Processor
↳ Peripheral Interrupt Enable

Summary
1) PIF → give allow to specific Peripheral to make Interrupt.
2) PIF → store the event state (it's fire or not)
3) GIE → give allow to Interrupt concept is work

* Interrupt type

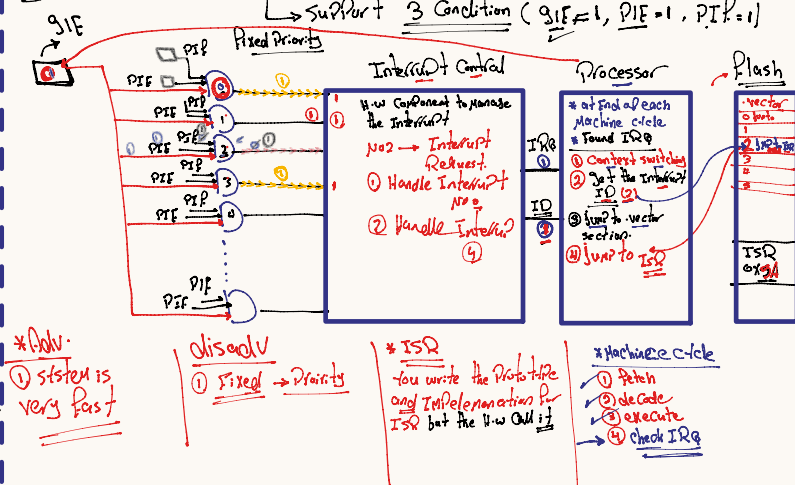
↳ according to Masking
Interrupt Maskable
As processor can be ignore allow PIF = 0
Interrupt - Non Maskable
* Interrupt the processor can't be ignore.
* Not have PIF → like Reset
Can't be ignore because not have PIF and not effect by Global Interrupt.

* Interrupt Handling

- 1) vectorable Priority
↳ ARM
- 2) Flexible Priority
↳ iC
- 3) NVIC :: nested vector Interrupt Control
↳ ARM

1) vectorable Priority

(Atmega32) Support 3 Condition (GIE=1, PIF=1, PIF=1)



* Adv.

- 1) system is very fast

disadv

- 1) Fixed → Priority

* ISR

you write the prototype and implementation for ISR but the H.W call it

* Machine cycle

- 1) Fetch
- 2) decode
- 3) execute
- 4) check IRQ

* Nested Interrupt

* Atmega32 Not support nested Interrupt



* what happen when Interrupt happen? → IC → IRQ (1)

- 1) Complete execution of current assembly instruction (by H.W)
- 2) Disable for global Interrupt → (by H.W)
- 3) Save program Counter (PC) (by H.W)
- 4) Save FPR (stack pointer, PSW, Acc, 2) (by S.W) → Compiler
- 5) Save GPR (by S.W) → Compiler
- 6) Jump to vector table (by S.W) → Compiler
- 7) Jump to ISR & execute it. (For yourself → just ISR start executed)
Flags clear by H.W
- 8) Enable global Interrupt → Based on Compiler (avr-gcc)
- 9) load the GPR → S.W
- 10) load the FPR → S.W
- 11) load the PC → H.W
- 12) Jump the continue Interrupted code.

Context switching
* store for some information about processor status to back again after finish
* store in SRAM

* Context switching

store & load some information from stack to move from code to another and back again (PC, SP, Acc, PSW, GPR)
type
1) H.W context switch :: like Interrupt → Call for another code by H.W.
↳ Some information store & load by H.W
2) S.W context switch :: like Call function.
↳ Context switch happen by S.W
↳ Compiler → Add the instruction of context switch

* Interrupt type

↳ according to Handle
1) vectorable
2) Flexible
3) NVIC

* Interrupt type

↳ according position

① outside Processor → and peripheral inside MCU can be interrupt
↳ ADC, OPB, CTC, UART Tx, UART Rx

② Inside Processor:

① traps
↳ fetch wrong Address.
↳ Decode wrong op code.
↳ execute: wrong logic
↳ interrupt his self to solve this issue.

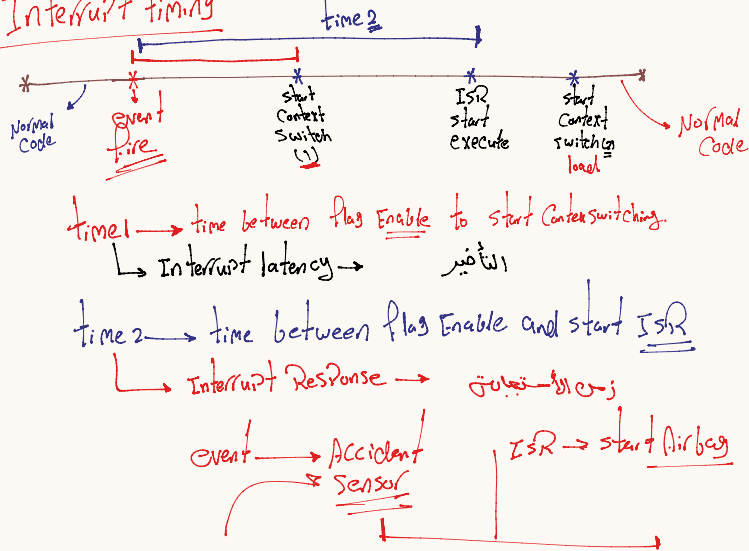
② SWI: Instruction Assembly → Call Function

③ Core Peripheral → Based on ARM

↳ inside processor

↳ SysTick, NVIC (inside processor)

Interrupt timing



* Event Handling in SW

① Polling ⇒ waiting

↳ wait until flag set

↳ flag will be set when the event fire

Processor → stop system
↳ wait flag

while (flag == 0); ←

* Busy wait

* Block system

② Interrupt

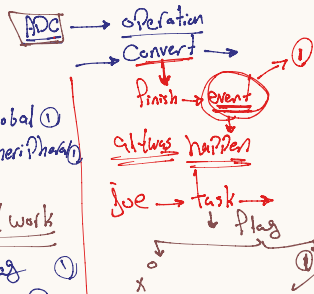
↳ Enable for global ①

↳ Enable for peripheral (ADC)

↳ Complete my work

↳ HW set flag ①

↳ jump to ISR



* How clear flag

↳ Polling

* Must be clear flag manual (SW)

① clear flag by write logic zero

② clear flag by write one (Atmega32)

Read flag

Write flag

3.0 VPP

Read flag

Write flag

Read flag

Write flag

Read flag

Write flag

Read flag

Write flag

Read flag

Write flag

Read flag

Write flag

Read flag

Write flag

Read flag

Write flag

Read flag

Write flag

↳ Interrupt

* Auto clear when start the ISR execution

* almost flags in Atmega cleared by HW when use Interrupt

* ISR → How write ISR Function Based on target

↳ Function

void __vector_no(void) __attribute__((signal));

void __vector_no(void)

↳

Independence

void __vector_no(void)

↳

Connect between this function and vector table section linker

Interrupt ID

vector table from data sheet

1

1

1

1

1

1

1

__attribute__((signal));

↳ Communicate with compiler or linker

↳ this function will be called by HW

Processor	MCU
PIC	16F877A
AVR	Atmega32
ARM	Tiva-C STM32F103